

Graphics Card Market Analysis - Web Scraping Project

Project Overview

In this project, you will create a web scraper to collect and analyze graphics card data from Newegg.com's GPU section. You'll use Python, BeautifulSoup, and other relevant libraries to extract key information about graphics cards currently available in the market.

Learning Objectives

- Implement web scraping techniques using Python and BeautifulSoup
- Handle pagination in web scraping
- Process and structure collected data
- Practice working with real-world data
- Learn to respect website scraping policies and implement delays

Part 1: Web scrapping

Required Data to Extract

For each graphics card listing, you must collect:

1. Product Image URL
2. Product Title (e.g., "MSI Ventus GeForce RTX 3060 12GB")
3. Product Details:
 - Manufacturer
 - Model Number
 - Rating (number of stars)
4. Pricing Information:
 - Current Price
 - Original Price (if available)
 - Shipping Information

 Screenshot

Project Requirements

Technical Requirements

1. Use Python 3.x
2. Required libraries:
 - requests
 - beautifulsoup4
 - pandas
 - time (for implementing delays)

Functional Requirements

1. Implement proper error handling
2. Include appropriate delays between requests (minimum 2 seconds)
3. Save the scraped data to a CSV file with appropriate column headers
4. Document your code with comments
5. Create a summary of the collected data (e.g., average price, price range)

Part 2: Data Analysis

Required Analysis

1. Price Analysis
 - Calculate average price by brand
 - Identify price ranges and market segments
 - Analyze price-to-memory ratio
 - Compare prices across different GPU series (RTX 3000 vs 4000)
 - Create price distribution visualizations
2. Brand Analysis
 - Market share by brand
 - Average ratings by brand
 - Price comparison across brands
 - Create brand comparison visualizations
3. Memory Configuration Analysis
 - Distribution of memory sizes
 - Price correlation with memory size
 - Popular memory configurations by brand
4. Market Segmentation
 - Entry-level vs Mid-range vs High-end segments
 - Price brackets and their features

Deliverables

1. Python script containing the web scraper
2. CSV file with the collected data
3. README file explaining:
 - Project Description
 - Features
4. Short report analyzing the collected data

Important Notes

- Always check and respect the website's robots.txt file
- Implement appropriate delays between requests to avoid overwhelming the server
- Handle your requests responsibly and ethically
- Be prepared to handle cases where data might be missing or in an unexpected format

```
In [20]: from selenium import webdriver
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import pandas as pd
import time
from selenium.webdriver.chrome.options import Options
from tqdm import tqdm
import requests
```

```
In [3]: url = "https://www.newegg.com/Desktop-Graphics-Cards/SubCategory/ID-48"
PAGES_TO_SCRAPE = 20
service = Service(ChromeDriverManager().install())
options = Options()
options.add_argument("--headless=new")
driver = webdriver.Chrome(options=options)
driver.get(url)
time.sleep(5)

gpu_list = []
page = 1

while page <= PAGES_TO_SCRAPE:
    driver.get(url + '/Page-' + str(page))

    products = container = driver.find_elements(By.CLASS_NAME, 'item-cell')

    for product in tqdm(products, 'in progress'):

        manu = product.find_elements(By.CLASS_NAME, 'item-brand')
        if manu:
            manu = manu[0].find_element(By.TAG_NAME, 'img').get_attribute('title')
        else:
            manu = None
```

```
ratings = product.find_elements(By.CLASS_NAME, 'item-rating')
if ratings:
    ratings = ratings[0].get_attribute('title').split('+ ', 1)[-1]
else:
    ratings = None

reviews = product.find_elements(By.CLASS_NAME, 'item-rating-num')
if reviews:
    reviews = reviews[0].text
else:
    reviews = None

img = product.find_elements(By.TAG_NAME, 'img')
if img:
    img_url = img[0].get_attribute('src')
else:
    img_url = None

ptil = product.find_elements(By.CLASS_NAME, 'item-title')
if ptil:
    ptil = ptil[0].text
else:
    ptil = None

model = product.find_elements(By.CLASS_NAME, 'item-features')
if model:
    model = model[0].text.split(':', 1)[-1]
else:
    model = None

price = product.find_elements(By.CLASS_NAME, 'price-current')
if price:
    price = price[0].text
else:
    price = None

ship = product.find_elements(By.CLASS_NAME, 'price-ship')
if ship:
    ship = ship[0].text
else:
    ship = None

was = product.find_elements(By.CLASS_NAME, 'price-was')
if was:
    was = was[0].text
else:
    was = None

gpu_data = {
    'image' : img_url,
    'manufacturer' : manu,
    'ratings' : ratings,
    'reviews' : reviews,
    'product' : ptil,
    'model' : model,
    'price' : price,
```

```
in progress: 100%|██████████| 36/36 [00:07<00:00, 4.93it/s]
in progress: 100%|██████████| 36/36 [00:07<00:00, 5.05it/s]
in progress: 100%|██████████| 36/36 [00:08<00:00, 4.21it/s]
in progress: 100%|██████████| 36/36 [00:07<00:00, 4.73it/s]
in progress: 100%|██████████| 36/36 [00:07<00:00, 4.86it/s]
in progress: 100%|██████████| 36/36 [00:06<00:00, 5.41it/s]
in progress: 100%|██████████| 36/36 [00:07<00:00, 5.00it/s]
in progress: 100%|██████████| 36/36 [00:07<00:00, 4.81it/s]
in progress: 100%|██████████| 36/36 [00:07<00:00, 4.59it/s]
in progress: 100%|██████████| 36/36 [00:07<00:00, 4.87it/s]
in progress: 100%|██████████| 36/36 [00:06<00:00, 5.33it/s]
in progress: 100%|██████████| 36/36 [00:07<00:00, 4.77it/s]
in progress: 100%|██████████| 36/36 [00:07<00:00, 4.83it/s]
in progress: 100%|██████████| 36/36 [00:07<00:00, 4.53it/s]
in progress: 100%|██████████| 36/36 [00:07<00:00, 5.13it/s]
in progress: 100%|██████████| 36/36 [00:07<00:00, 4.58it/s]
in progress: 100%|██████████| 36/36 [00:07<00:00, 5.10it/s]
in progress: 100%|██████████| 36/36 [00:07<00:00, 4.77it/s]
in progress: 100%|██████████| 36/36 [00:07<00:00, 4.51it/s]
in progress: 100%|██████████| 36/36 [00:08<00:00, 4.37it/s]
```

localhost:8888/lab/tree/Newegg Project/Newegg Webscraping Project.ipynb

Part 2: Data Analysis

In [22]:

import matplotlib.pyplot as plt

In [23]:

df.head()

Out[23]:

	image	manufacturer	ratings	reviews	prod
0	https://c1.neweggimages.com/productimage/nb300...	Sapphire Tech	4.7	(124)	SAPPHIRE PULSE Rad RX 9 XT 10 GDI I
1	https://c1.neweggimages.com/productimage/nb300...	ASRock	4.7	(109)	ASRock Challenger Rad RX 9 XT 10 GDDI
2	https://c1.neweggimages.com/productimage/nb300...	GIGABYTE	4.8	(241)	GIGABYTE Gaming GeForce RTX 5 Ti 10 GDDI
3	https://c1.neweggimages.com/productimage/nb300...	GIGABYTE	4.6	(300)	GIGABYTE Gaming GeForce RTX 5 10 GDI I
4	https://c1.neweggimages.com/productimage/nb300...	ASRock	4.6	(139)	ASRock Challenger Rad RX 9 XT 10 GDDI

Required Analysis

Price Analysis

```
In [24]: df = df.replace('',pd.NA).replace('COMING SOON', pd.NA)
```

```
In [25]: df = df.dropna(subset='price')
```

```
In [26]: df['price'] = df['price'].str.replace('$','').str.replace(',','').astype('float')
```

```
In [27]: avg_price = df.groupby('manufacturer')['price'].mean()
avg_price
```

```
Out[27]: manufacturer
AMD                304.325000
ASRock             481.206333
ASUS               1159.934936
GIGABYTE           871.471587
HP                 809.000000
Intel              504.500000
Lenovo             236.078000
MSI                942.778596
NVIDIA            1108.506078
PowerColor         697.124500
Sapphire Tech      824.375714
XFX                482.810000
ZOTAC              1169.876563
Name: price, dtype: float64
```

```
In [28]: df['price'].describe()
```

```
Out[28]: count      717.000000
mean        872.912678
std         915.964895
min          18.740000
25%         339.990000
50%         599.000000
75%        1079.000000
max         6999.000000
Name: price, dtype: float64
```

```
In [29]: bins = [0,1200,4000,float('inf')]
labels = ['Entry-level', 'Mid-range', 'High-end']
```

```
In [30]: df['segment'] = pd.cut(df['price'],bins=bins,labels=labels)
df['segment'].value_counts()
```

```
Out[30]: segment
Entry-level    561
Mid-range      142
High-end       14
Name: count, dtype: int64
```

```
In [36]: print(df.columns.tolist())
```

```
['image', 'manufacturer', 'ratings', 'reviews', 'product', 'model', 'price', 'shipping_cost', 'original_price', 'segment']
```

```
In [39]: import re

def extract_memory(title):
    if not isinstance(title, str):
        return None
    gb_match = re.search(r'(\d+)\s?GB(?:[a-zA-Z])?|(\d+)\s?G(?:[a-zA-Z])?', title)
    if gb_match:
        return float(gb_match.group(1) or gb_match.group(2))
    mb_match = re.search(r'(\d+)\s?MB(?:[a-zA-Z])?', title)
    if mb_match:
        return float(mb_match.group(1)) / 1024
    return None

df['memory_gb'] = df['product'].apply(extract_memory)
df.loc[df['memory_gb'] > 48, 'memory_gb'] = None
df = df.dropna(subset=['memory_gb'])
```

```
In [40]: df['price_per_gb'] = df['price'] / df['memory_gb']
df['price_per_gb']
```

```
Out[40]: 0      32.499375
1      29.374375
2      67.999375
3     106.249375
4      44.999375
...
715    32.498333
716    112.437500
717     88.250000
718    155.000000
719     29.655625
Name: price_per_gb, Length: 706, dtype: float64
```

```
In [42]: df['series'] = df['product'].str.extract(r'RTX\s?(\d{4})', expand=False)
df['series']
```

```
Out[42]: 0      NaN
1      NaN
2     5070
3     5080
4      NaN
...
715    NaN
716    5070
717    4070
718    NaN
719    NaN
Name: series, Length: 706, dtype: object
```

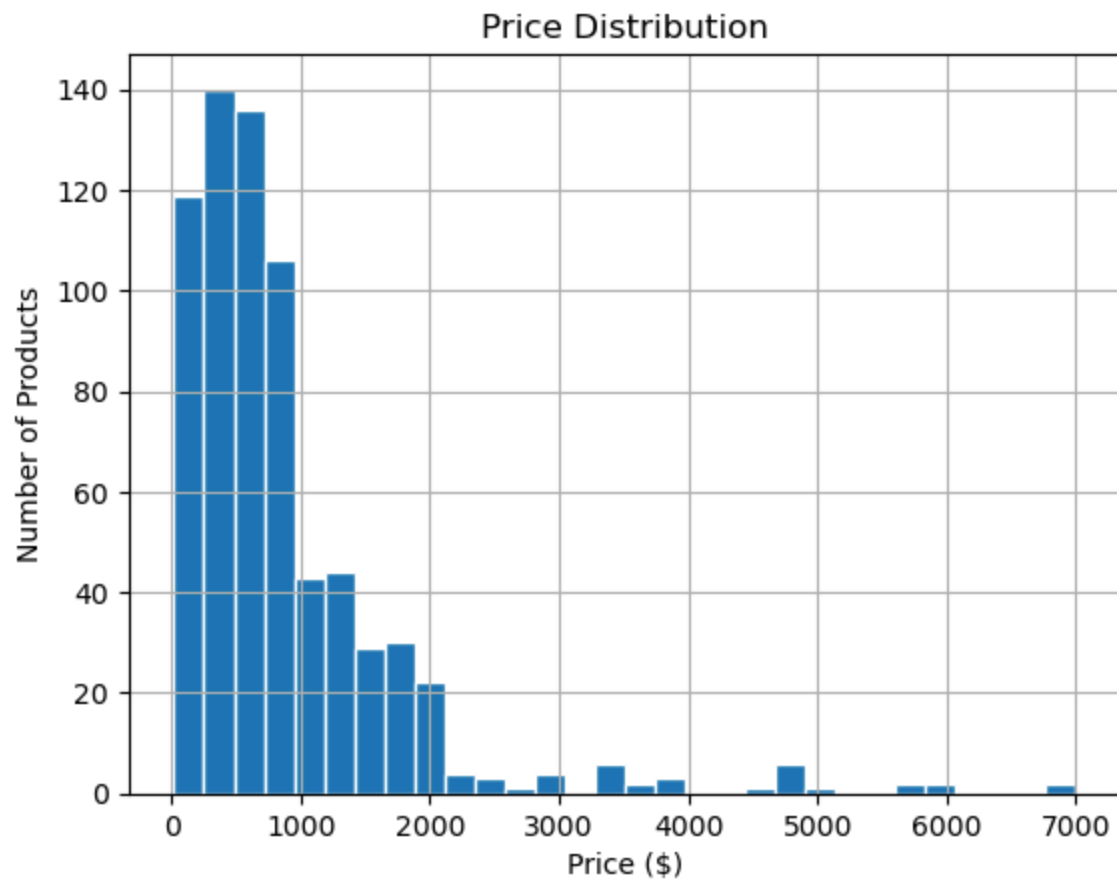
```
In [43]: df.groupby('series')['price'].describe()
```

Out[43]:

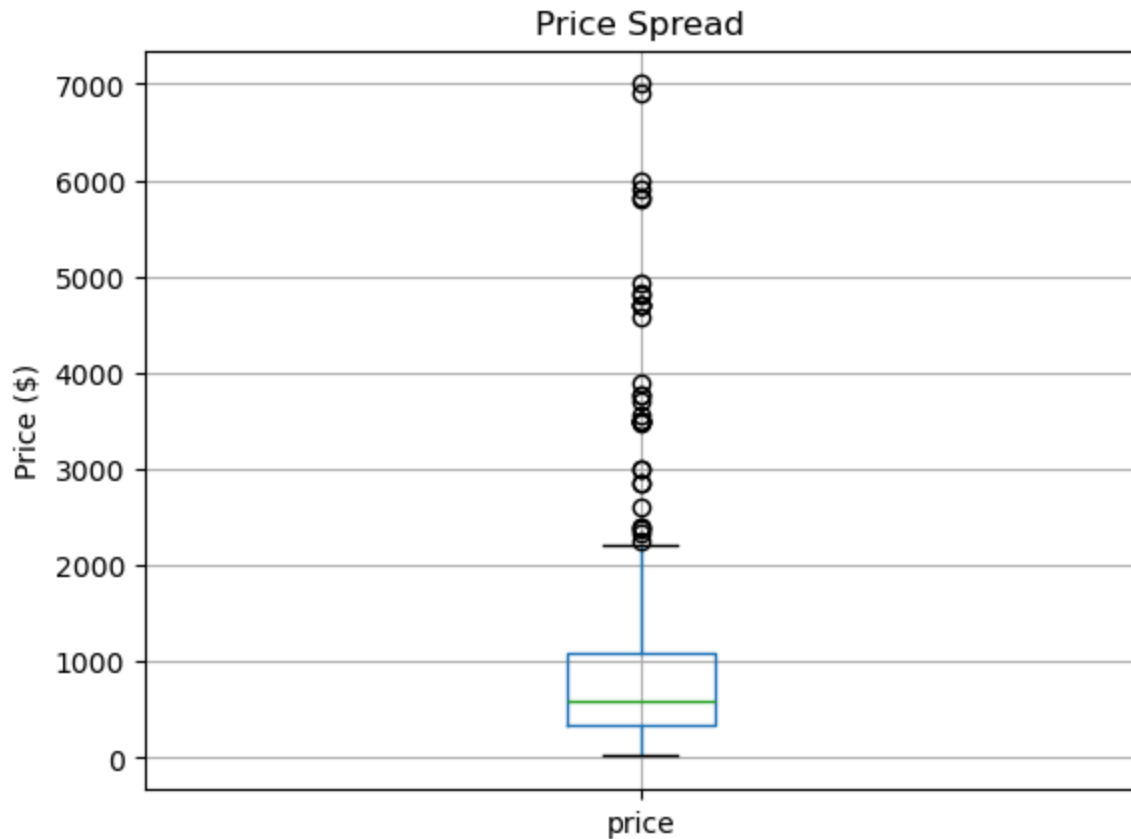
	count	mean	std	min	25%	50%	75%	max
series								
2000	2.0	899.000000	141.421356	799.00	849.000	899.000	949.0000	999.00
2060	3.0	411.500000	169.977204	267.50	317.750	368.000	483.5000	599.00
2070	2.0	580.000000	120.208153	495.00	537.500	580.000	622.5000	665.00
2080	4.0	708.250000	21.250490	695.00	698.000	699.000	709.2500	740.00
3050	17.0	290.447647	72.814589	199.99	229.000	279.850	349.0000	425.90
3060	32.0	523.202188	90.899726	319.99	483.020	498.995	599.0000	695.00
3070	13.0	552.254615	183.216739	305.00	389.990	549.990	735.0000	795.00
3080	13.0	770.346923	263.762120	429.99	599.990	749.990	849.0000	1289.00
3090	12.0	1976.831667	228.878042	1499.99	1980.000	2099.000	2099.0000	2099.00
4000	1.0	2250.000000	NaN	2250.00	2250.000	2250.000	2250.0000	2250.00
4060	19.0	563.818421	170.270230	339.99	424.495	566.400	659.5900	852.63
4070	40.0	1048.017500	273.724787	529.00	844.990	1029.610	1273.8875	1758.00
4080	20.0	1654.247500	203.965856	1149.99	1594.000	1649.000	1788.3425	2036.99
4090	10.0	3566.099000	246.551374	2999.99	3496.000	3532.000	3748.5000	3895.00
5000	4.0	1777.250000	1868.509277	835.00	842.500	847.000	1781.7500	4580.00
5050	4.0	393.242500	156.877282	299.99	314.990	322.490	400.7425	628.00
5060	58.0	642.822931	183.109464	459.99	502.490	545.445	789.9100	1079.00
5070	63.0	1179.233810	300.683141	699.99	924.990	1169.990	1382.6100	1899.99
5080	37.0	1954.448649	473.882631	1399.99	1694.990	1799.990	2099.9900	3499.00
5090	13.0	5445.083846	846.499060	4699.99	4699.990	4929.990	5899.9900	6999.00

In [44]:

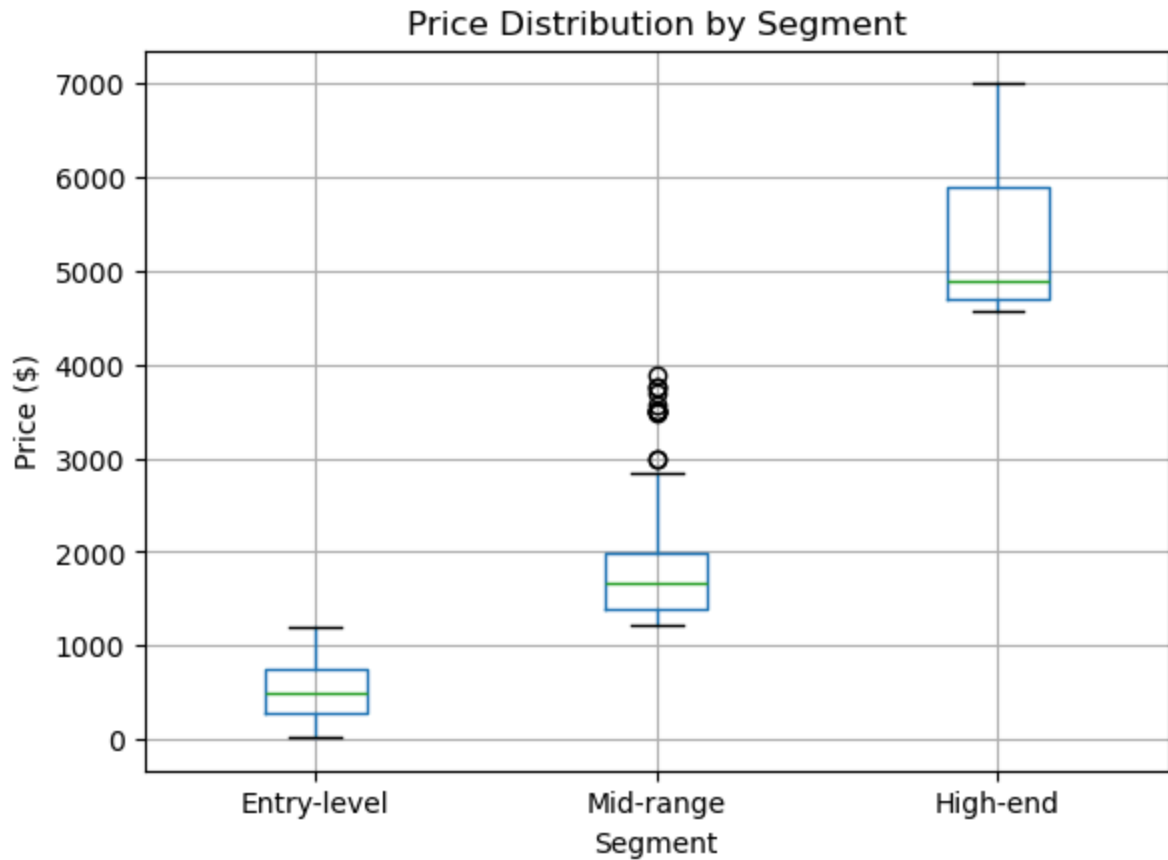
```
df['price'].hist(bins=30, edgecolor='white')
plt.xlabel('Price ($)')
plt.ylabel('Number of Products')
plt.title('Price Distribution')
plt.show()
```



```
In [45]: df.boxplot(column='price')  
plt.ylabel('Price ($)')  
plt.title('Price Spread')  
plt.show()
```



```
In [46]: df.boxplot(column='price', by='segment')
plt.xlabel('Segment')
plt.ylabel('Price ($)')
plt.title('Price Distribution by Segment')
plt.suptitle('')
plt.show()
```



Brand Analysis

```
In [47]: brand_counts = df['manufacturer'].value_counts()
brand_share = (brand_counts / brand_counts.sum() * 100).round(2)
brand_share
```

```
Out[47]: manufacturer
ASUS                26.40
GIGABYTE            21.15
MSI                 19.29
NVIDIA              8.29
Sapphire Tech       5.75
ZOTAC               5.25
ASRock              4.91
XFX                 3.55
PowerColor          3.38
AMD                 1.02
Lenovo              0.68
Intel               0.17
HP                  0.17
Name: count, dtype: float64
```

```
In [50]: df['ratings'] = pd.to_numeric(df['ratings'])
df.groupby('manufacturer')['ratings'].mean().sort_values(ascending=False)
```

```
Out[50]: manufacturer
AMD          5.000000
Lenovo       5.000000
NVIDIA       4.687500
XFX          4.669231
ASUS         4.635227
MSI          4.553535
GIGABYTE     4.544340
Sapphire Tech 4.525000
ASRock       4.373913
ZOTAC        4.346667
PowerColor   4.300000
HP           NaN
Intel        NaN
Name: ratings, dtype: float64
```

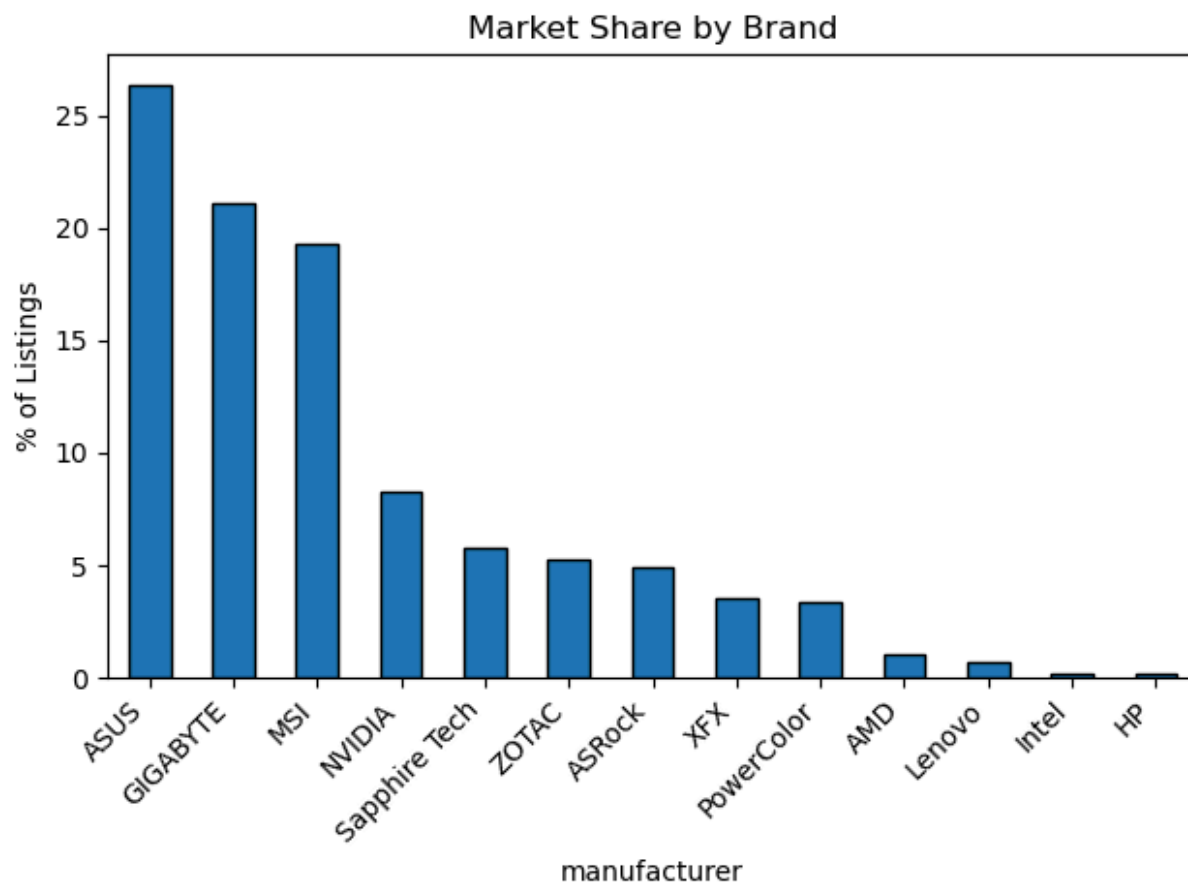
```
In [51]: df.groupby('manufacturer')['price'].describe().sort_values('mean', ascending=False)
```

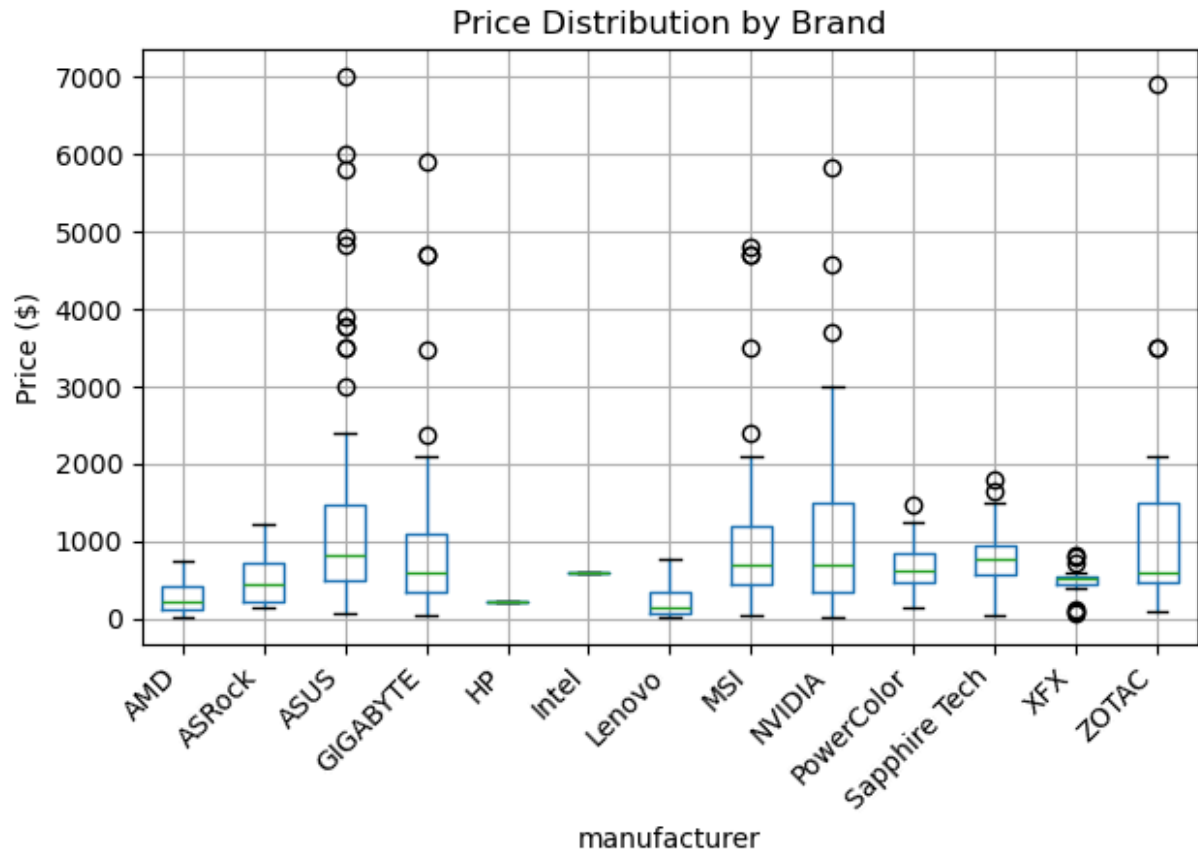
```
Out[51]:
```

	count	mean	std	min	25%	50%	75%	m
manufacturer								
ZOTAC	31.0	1198.614516	1356.854774	99.99	469.9900	599.990	1490.6200	6900.
ASUS	156.0	1159.934936	1126.658143	68.99	487.4900	814.990	1484.7950	6999.
NVIDIA	49.0	1131.813673	1222.306130	33.25	354.9900	706.560	1495.0000	5827.
MSI	114.0	942.778596	861.303194	44.99	445.0000	697.495	1199.9900	4799.
GIGABYTE	125.0	875.843360	879.325811	39.99	349.0000	599.000	1087.9900	5899.
Sapphire Tech	34.0	843.916471	405.161998	56.00	567.9975	782.990	957.1250	1799.
PowerColor	20.0	697.124500	314.217641	144.99	476.7475	617.490	839.9900	1480.
Intel	1.0	590.000000	NaN	590.00	590.0000	590.000	590.0000	590.
ASRock	29.0	490.558621	289.211201	139.99	220.4800	445.000	719.9900	1215.
XFX	21.0	489.610952	200.807060	59.99	449.9900	509.990	554.9900	829.
AMD	6.0	304.325000	270.770134	22.99	131.7400	224.990	429.9900	754.
Lenovo	4.0	275.097500	349.987701	18.74	63.9350	148.325	359.4875	785.
HP	1.0	219.000000	NaN	219.00	219.0000	219.000	219.0000	219.

```
In [52]: # Market share
brand_share.plot(kind='bar', edgecolor='black')
plt.ylabel('% of Listings')
plt.title('Market Share by Brand')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
# Price by brand
df.boxplot(column='price', by='manufacturer')
plt.xticks(rotation=45, ha='right')
plt.ylabel('Price ($)')
plt.title('Price Distribution by Brand')
plt.suptitle('')
plt.tight_layout()
plt.show()
```





Memory Configuration Analysis

```
In [53]: df['memory_gb'].value_counts().sort_index()
```

```
Out[53]: memory_gb
0.25      1
0.50      2
1.00     13
1.50      1
2.00     34
3.00      5
4.00     51
5.00      3
6.00     31
8.00    153
10.00     11
11.00      7
12.00    108
16.00    221
20.00      9
24.00     41
32.00     15
Name: count, dtype: int64
```

```
In [54]: print(df['price'].corr(df['memory_gb']).round(2))
```

```
0.77
```

```
In [55]: df.groupby(['manufacturer', 'memory_gb']).size().unstack(fill_value=0)
```

Out[55]:

memory_gb	1.0	4.0	8.0	12.0	16.0	6.0	10.0	20.0	24.0	2.0	3.0	5.0	11.0	32.0
manufacturer														
AMD	1	1	1	2	1	0	0	0	0	0	0	0	0	0
ASRock	0	1	9	4	9	2	1	2	1	0	0	0	0	0
ASUS	1	5	33	28	54	7	2	1	10	6	1	2	1	5
GIGABYTE	0	10	33	19	38	6	1	0	4	8	1	0	2	3
HP	0	0	0	0	0	0	0	0	0	1	0	0	0	0
Intel	0	0	0	1	0	0	0	0	0	0	0	0	0	0
Lenovo	1	1	1	0	1	0	0	0	0	0	0	0	0	0
MSI	4	5	28	21	36	6	2	0	3	5	0	0	1	3
NVIDIA	1	2	10	5	13	0	1	2	10	1	0	0	1	3
PowerColor	0	2	1	4	12	0	0	0	1	0	0	0	0	0
Sapphire Tech	0	2	3	3	15	0	0	3	7	1	0	0	0	0
XFX	0	1	8	2	10	0	0	0	0	0	0	0	0	0
ZOTAC	0	1	9	4	9	1	2	0	3	1	0	0	0	1

```
In [56]: df.groupby('segment', observed=False)['price'].describe()
```

Out[56]:

	count	mean	std	min	25%	50%	75%	max
segment								
Entry-level	551.0	520.225426	296.122747	18.74	285.975	499.00	751.995	1199.99
Mid-range	141.0	1835.334610	632.550141	1215.00	1399.990	1659.99	1995.000	3895.00
High-end	14.0	5383.292143	845.514998	4580.00	4699.990	4879.99	5881.790	6999.00

```
In [57]: df.groupby('segment', observed=False)['memory_gb'].mean()
```

Out[57]:

segment	
Entry-level	9.848911
Mid-range	17.843972
High-end	32.000000
Name: memory_gb, dtype: float64	